Quantization-Induced Formatting Drift: Empirical Analysis of Structural Schema Collapse in Small Language Models

Y. Vedartha Sai

Abstract—Small Language Models (SLMs, $\leq$ 3B parameters) are increasingly integrated into enterprise software architectures, microservice backends, and edge computing platforms where rigid JSON schema compliance is an absolute operational requirement. To reduce computational footprint and memory bandwidth requirements during CPU deployment, Post-Training Quantization (PTQ)—particularly dynamic INT8 quantization—is widely adopted. However, while quantization effects on semantic quality have been broadly studied, its impact on deterministic structural syntax integrity—termed Formatting Drift ($D_{\text{formatting}}$)—remains insufficiently quantified. In this paper, we conduct an exhaustive empirical investigation into structural schema collapse in the Qwen2.5-Instruct model family (Qwen2.5-0.5B-Instruct and Qwen2.5-1.5B-Instruct) under dynamic INT8 CPU quantization. Utilizing a stratified benchmark dataset of 100 structured extraction tasks divided across three complexity tiers (Primitive Flat, Nested Object, and Typed Array), we observe catastrophic structural formatting collapse. Dynamic INT8 quantization induces a relative formatting drift of -93.0% in the 0.5B model ($p < 10^{-15}$) and a complete -100.0% failure in the 1.5B model ($p < 10^{-15}$). Contrary to the parameter capacity buffering hypothesis, scaling parameter capacity from 0.5B to 1.5B fails to mitigate formatting collapse. Diagnostic evaluation across 193 failure instances reveals a systematic error distribution: 67.9% stem from unparseable syntax errors (SYNTAX_ERROR), dominated by conversational preambles and markdown code fence artifacts, while 31.6% stem from schema field mismatches (SCHEMA_FIELD_MISMATCH), characterized by raw schema definition echoing. Mathematical modeling demonstrates that dynamic per-tensor quantization clips activation outliers in self-attention projection layers, systematically eroding the logit margin between structural control tokens and natural language conversational tokens. We conclude that unconstrained dynamic INT8 quantization is fundamentally unsuitable for structured data pipelines in small language models, highlighting the necessity of grammar-constrained decoding or outlier-aware block-wise quantization.

Index Terms—Small Language Models, Post-Training Quantization, Formatting Drift, JSON Schema Compliance, Structural Failure Taxonomy, Attention Outliers, Edge Computing, Microservice Reliability.

1 Introduction

The rapid deployment of Small Language Models (SLMs, $\leq$ 3B parameters) into production software backends, autonomous agent workflows, and edge computing environments has reshaped artificial intelligence infrastructure. In modern agentic pipelines, language models function not merely as conversational chatbots, but as structured information extraction engines that translate unstructured natural language into machine-readable payloads, including JSON documents, XML trees, SQL database queries, and API function call invocations [?], [?]. These enterprise integration frameworks operate under a fundamental assumption: the model's generated text must strictly adhere to predefined formal syntax specifications and typed data schemas.

In production microservices, schema compliance is binary and unforgiving. A single syntax violation—such as a missing quotation mark, an unclosed bracket, a trailing comma, conversational preamble text (e.g., "Here is the JSON object you requested:"), or raw schema definition echoing—causes immediate parsing exceptions in downstream JSON serialization pipelines. Such structural failures result in broken workflows, unhandled application errors, or costly fallback routines that negate the efficiency gains of deploying lightweight models.

To enable execution on resource-constrained hardware such as CPU servers, mobile devices, and embedded edge systems, model quantization is standard industry practice [?], [?]. Post-Training Quantization (PTQ) maps high-precision floating-point model weights and activations (e.g., 16-bit Floating Point, FP16) to low-bit integer representations (e.g., 8-bit Integer, INT8). Dynamic INT8 CPU quantization yields up to $4\times$ memory reduction and substantial inference throughput acceleration by utilizing vector instruction sets such as Intel AVX-512 and VNNI.

While prior research has extensively benchmarked quantization across general academic benchmarks such as MMLU, GSM8K, and HumanEval [?], [?], these evaluations focus almost exclusively on high-level semantic reasoning or task accuracy. Standard evaluation protocols typically rely on flexible regular expression extraction or liberal string matching, effectively masking structural syntax failures. Consequently, the impact of low-bit quantization on strict syntactic schema integrity and structural formatting stability remains a critical unaddressed vulnerability in software engineering literature.

In this work, we address this critical gap by establishing a formal theoretical and empirical framework to measure

• Y. Vedartha Sai is with the Department of Information Technology, Indian Institute of Information Technology Allahabad, Prayagraj 211015, Uttar Pradesh, India.
E-mail: mhm2026013@iiita.ac.in

Manuscript received October 15, 2026; revised October 28, 2026.

Formatting Drift ($D_{\text{formatting}}$)—the relative degradation in deterministic structural schema compliance caused by precision reduction. We perform an empirical evaluation on the state-of-the-art Qwen2.5-Instruct model family [?] at two parameter scales: Qwen2.5-0.5B-Instruct and Qwen2.5-1.5B-Instruct. We evaluate these models across a controlled experimental matrix comparing unquantized FP16 baselines against dynamic per-tensor INT8 CPU quantization under strict greedy decoding ($T = 0.0$, locked random seed).

1.1 Research Questions

Our study is structured around three primary research questions:

- RQ1 (Quantization-Induced Formatting Drift): Does post-training dynamic INT8 CPU quantization induce a statistically significant degradation in structural JSON schema compliance compared to FP16 baselines?
- RQ2 (Parameter Capacity Buffering Hypothesis): Does increasing model parameter capacity from 0.5B to 1.5B buffer the architecture against quantization-induced structural formatting collapse?
- RQ3 (Failure Dynamics & Structural Complexity): How does formatting drift vary across structural complexity tiers (Primitive Flat, Nested Object, and Typed Array), and what are the dominant syntactic failure categories?

1.2 Key Contributions

The primary contributions of this paper are summarized as follows:

1) Formalization of Formatting Drift Metric: We introduce $D_{\text{formatting}}$ alongside a dual-pass verification framework that rigorously decouples strict raw JSON syntax compliance (P_{raw}) from scrubbed markdown compliance (P_{scrubbed}), establishing a standardized benchmark methodology for structural reliability.
2) Empirical Refutation of Capacity Buffering: We demonstrate that uncalibrated dynamic INT8 quantization triggers catastrophic structural collapse in both 0.5B ($D_{\text{formatting}} = -93.0\%$) and 1.5B ($D_{\text{formatting}} = -100.0\%$) models ($p < 10^{-15}$ for both), disproving the assumption that increasing parameter capacity mitigates formatting degradation.
3) Diagnostic Failure Taxonomy: We classify 193 structural failure instances into a three-tiered taxonomy: SYNTAX_ERROR (67.9%), SCHEMA_FIELD_MISMATCH (31.6%), and SCHEMA_TYPE_MISMATCH (0.5%), identifying conversational chatter and raw schema definition echoing as the leading failure mechanisms.
4) Attention Outlier Mechanism Analysis: We derive the mathematical mechanism underlying this collapse: dynamic uniform quantization clips activation outliers in self-attention projection layers, eroding the logit margin between structural control tokens (e.g., braces, quotes, colons) and natural language text tokens.
5) Actionable Architectural Blueprint: We rank three mitigation strategies by computational cost and information gain, providing clear deployment guidance for reliable SLM microservice engineering.

2 Theoretical Foundations & Related Work

2.1 Post-Training Quantization Frameworks

Post-Training Quantization (PTQ) compresses neural network parameters without requiring full model retraining or fine-tuning datasets [?]. In uniform affine quantization, a real-valued floating-point tensor $X \in \mathbb{R}$ is mapped to a discrete low-bit integer representation $X_q \in [\text{qmin}, \text{qmax}]$ via a positive scalar scale factor $S \in \mathbb{R}^+$ and an integer zero-point $Z \in \mathbb{Z}$:

$$X_q = \text{clamp}\left(\left\lfloor \frac{X}{S} \right\rceil + Z, \text{qmin}, \text{qmax}\right) \tag{1}$$

where $\lfloor \cdot \rceil$ denotes round-to-nearest integer rounding, and $\text{clamp}(v, a, b) = \max(a, \min(b, v))$. Reconstruction of the approximate floating-point values $\hat{X} \approx X$ (dequantization) is expressed as:

$$\hat{X} = S \cdot (X_q - Z) \tag{2}$$

In dynamic per-tensor INT8 quantization (e.g., PyTorch 'quantize$_d ynamic$'), $linear layer weights \text{W} are quantized of f line per-channel or per-tensor, whereas activation matrices \text{H} are dynamically q tensor during execution. The dynamic scale factor \text{S}_H$ and zero-point Z_H are computed dynamically at runtime based on the instantaneous minimum and maximum values of the activation tensor:

$$S_H = \frac{\max(\mathbf{H}) - \min(\mathbf{H})}{\text{qmax} - \text{qmin}}, \quad Z_H = \left\lfloor \frac{-\min(\mathbf{H})}{S_H} \right\rceil + \text{qmin} \tag{3}$$

While dynamic INT8 execution significantly accelerates matrix multiplications on CPU architectures via SIMD integer vector instructions (e.g., AVX-512 VNNI), it introduces quantization error noise $\mathbf{E}_H = \mathbf{H} - \hat{\mathbf{H}}$. When activation distributions exhibit heavy-tailed outliers, calculating S_H across the full dynamic range $\max(\mathbf{H}) - \min(\mathbf{H})$ stretches the quantization bin width S_H, severely coarsening the representation of non-outlier activations [?], [?].

2.2 Activation Outliers and Attention Routing Dynamics

Dettmers et al. [?] first documented the emergence of systematic activation outliers in Transformer language models once model scale exceeds 6.7B parameters. However, recent architectural advancements in small language models—such as SwiGLU activation functions, Grouped Query Attention (GQA), Rotary Position Embeddings (RoPE), and intensive pre-training on multi-trillion token corpora—have dramatically altered internal feature representations even in sub-2B parameter models like Qwen2.5 [?].

Xiao et al. [?] established that activation outliers are highly concentrated in specific feature channels across attention projection matrices ($\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}_O$) and Feed-Forward Networks (FFN). When activations are quantized dynamically to INT8 without channel-wise smoothing (e.g., SmoothQuant) or outlier-aware decomposition (e.g., LLM.int8()), the extreme magnitude of these outlier channels dominates the dynamic range calculation in Eq. (??).

Consequently, the remaining 99.9% of normal activation channels suffer massive quantization noise, degrading the model's capacity to maintain precise attention routing over structural delimiter tokens.

2.3 Structured LLM Generation Paradigms

To enforce structured output generation in production environments, system architects rely on two distinct paradigms:

2.3.1 Unconstrained Decoding with Instruction Framing

The language model is provided with system prompts containing strict formatting instructions and JSON schema definitions. Output generation proceeds via standard unconstrained autoregressive decoding, relying entirely on the model's learned probability distribution $P(w_t \mid w_{<t})$ to emit valid syntax. While computationally lightweight and universally compatible across API providers, unconstrained generation is vulnerable to formatting drift, conversational chatter, and structural degradation when weights or activations are perturbed.

2.3.2 Schema-Constrained Decoding (Grammar-Guided Generation)

Frameworks such as Outlines [?], Guidance, and llama.cpp grammar masking construct a Context-Free Grammar (CFG) or Deterministic Finite Automaton (DFA) from the target JSON schema. At each decoding step t, the state machine determines the set of valid next tokens $V_{\text{valid}} \subseteq V$. Tokens outside V_{valid} are masked by setting their logits to $-\infty$ prior to softmax evaluation:

$$z'_{t,k} = \begin{cases} z_{t,k}, & \text{if } w_k \in V_{\text{valid}} \\ -\infty, & \text{otherwise} \end{cases} \tag{4}$$

While schema-constrained decoding guarantees 100% syntactic schema compliance, it introduces runtime state machine maintenance overhead and requires low-level access to decoding logit vectors, which is often unavailable in high-throughput CPU microservices or black-box API endpoints. Evaluating the inherent structural stability of quantized models under unconstrained decoding is therefore critical for understanding foundational model robustness.

3 Formatting Drift ($D_{\text{formatting}}$) Formalism

3.1 Mathematical Definition

Let $\mathcal{M}_{\text{FP16}}$ represent an unquantized baseline model operating in FP16 precision, and let $\mathcal{M}_{\text{INT8}}$ denote its quantized counterpart. Given a dataset $\mathcal{D} = \{(x_i, \mathcal{S}_i)\}_{i=1}^{N}$ comprising N natural language extraction prompts x_i paired with formal target JSON schemas $\mathcal{S}_i$, let $y_i = \mathcal{M}(x_i)$ represent the complete text sequence generated under deterministic greedy decoding ($T = 0.0$).

We define a binary verification operator $\mathcal{V}(y_i, \mathcal{S}_i) \in \{0, 1\}$ that evaluates whether output string y_i is a syntactically valid JSON payload that satisfies all mandatory keys, structural depth constraints, and primitive types specified by schema $\mathcal{S}_i$:

$$\mathcal{V}(y_i, \mathcal{S}_i) = \begin{cases} 1, & \text{if JSON.parse}(y_i) \text{ succeeds AND } y_i \models \mathcal{S}_i \\ 0, & \text{otherwise} \end{cases} \tag{5}$$

The overall structural pass rate P across dataset $\mathcal{D}$ is given by:

$$P = \frac{1}{N} \sum_{i=1}^{N} \mathcal{V}(y_i, \mathcal{S}_i) \tag{6}$$

Definition 1 (Relative Formatting Drift). The Relative Formatting Drift $D_{\text{formatting}}$ between an unquantized baseline model $\mathcal{M}_{\text{FP16}}$ and a quantized model $\mathcal{M}_{\text{INT8}}$ is defined as:

$$D_{\text{formatting}} = \left(\frac{P_{\text{INT8}} - P_{\text{FP16}}}{P_{\text{FP16}}} \right) \times 100\% \tag{7}$$

where $P_{\text{FP16}} > 0$.

When the FP16 baseline achieves complete structural compliance ($P_{\text{FP16}} = 1.0$), Eq. (??) simplifies directly to the percentage point change: $D_{\text{formatting}} = (P_{\text{INT8}} - 1.0) \times 100\%$.

3.2 Dual-Pass Verification Engine

To isolate unparseable syntax collapse from minor markdown formatting artifacts, we implement a dual-pass evaluation engine:

1) Strict Raw Pass Rate (P_{raw}): Evaluates the raw completion string y_i directly without modification. Any leading conversational preamble, trailing explanation, or markdown code fence wrapper (“‘json ... “‘) results in an immediate evaluation failure ($\mathcal{V}_{\text{raw}} = 0$).
2) Scrubbed Pass Rate (P_{scrubbed}): Applies a deterministic regex extraction operator $\mathcal{F}_{\text{scrub}}(y_i)$ to locate markdown code blocks or isolate the substring bounded by the outermost curly braces { and }. The extracted payload is then validated against schema $\mathcal{S}_i$:

$$P_{\text{scrubbed}} = \frac{1}{N} \sum_{i=1}^{N} \mathcal{V}(\mathcal{F}_{\text{scrub}}(y_i), \mathcal{S}_i) \tag{8}$$

The pass rate disparity $\Delta P = P_{\text{scrubbed}} - P_{\text{raw}}$ quantifies the prevalence of ”wrapper chatter”—instances where the model generated valid JSON data enclosed within conversational text wrappers.

3.3 Statistical Significance and Pooled Z-Test Derivations

To evaluate whether observed drops in structural pass rates are statistically significant, we apply a pooled two-proportion Z-test.

Let $x_1 = P_{\text{FP16}} \cdot N$ and $x_2 = P_{\text{INT8}} \cdot N$ represent the number of successful schema compliance outcomes across sample size $N = 100$. The pooled success proportion $\bar{p}$ is:

$$\bar{p} = \frac{x_1 + x_2}{2N} = \frac{P_{\text{FP16}} + P_{\text{INT8}}}{2} \tag{9}$$

The standard error of the difference between the two proportions is formulated as:

$$\text{SE} = \sqrt{\bar{p}(1 - \bar{p}) \left(\frac{2}{N} \right)} \tag{10}$$

The two-proportion test statistic Z is computed as:

$$Z = \frac{P_{\text{INT8}} - P_{\text{FP16}}}{\text{SE}} \tag{11}$$

The two-tailed p-value is determined via the cumulative distribution function $\Phi(\cdot)$ of the standard normal distribution:

$$p = 2 \cdot (1 - \Phi(|Z|)) \tag{12}$$

Theorem 1 (Variance Bound under Zero-Pass Collapse). When an unquantized model achieves perfect structural compliance ($P_{\text{FP16}} = 1.0$) and its quantized variant suffers total structural collapse ($P_{\text{INT8}} = 0.0$), the pooled proportion simplifies to $\bar{p} = 0.5$, and the test statistic achieves its minimum bound:

$$Z_{\text{bound}} = -\sqrt{2N} \tag{13}$$

Proof. Substituting $P_{\text{FP16}} = 1.0$ and $P_{\text{INT8}} = 0.0$ into Eq. (??) yields:

$$\bar{p} = \frac{1.0 + 0.0}{2} = 0.5 \tag{14}$$

Substituting $\bar{p} = 0.5$ into the standard error expression:

$$\text{SE} = \sqrt{0.5 \cdot (1 - 0.5) \cdot \frac{2}{N}} = \sqrt{0.25 \cdot \frac{2}{N}} = \sqrt{\frac{0.5}{N}} = \frac{1}{\sqrt{2N}} \tag{15}$$

Substituting $\text{SE} = 1/\sqrt{2N}$ into Eq. (??):

$$Z = \frac{0.0 - 1.0}{1/\sqrt{2N}} = -\sqrt{2N} \tag{16}$$

For sample size $N = 100$, $Z = -\sqrt{200} \approx -14.1421$, which corresponds to $p = 2 \cdot (1 - \Phi(14.1421)) \approx 1.6 \times 10^{-45} \ll 10^{-15}$. □

4 Mathematical Mechanism of Attention Routing Collapse

In this section, we derive the mathematical mechanism explaining why dynamic per-tensor INT8 quantization degrades structural formatting control tokens while leaving general language fluency largely intact.

4.1 Softmax Logit Margin Degradation

Let $\mathbf{h}_t \in \mathbb{R}^d$ represent the final-layer hidden state vector at decoding step t. In an unquantized FP16 network, the logit vector $\mathbf{z}_t \in \mathbb{R}^{|V|}$ over vocabulary V is computed via linear transformation with the language model head weight matrix $\mathbf{W}_{\text{head}} \in \mathbb{R}^{|V| \times d}$:

$$\mathbf{z}_t = \mathbf{W}_{\text{head}} \mathbf{h}_t \tag{17}$$

The probability distribution $P(w_k \mid x_{<t})$ over vocabulary tokens $w_k \in V$ is:

$$P(w_k \mid x_{<t}) = \frac{\exp(z_{t,k})}{\sum_{j \in V} \exp(z_{t,j})} \tag{18}$$

Under greedy decoding ($T = 0.0$), the model emits token w^*:

$$w^* = \arg\max_{k \in V} z_{t,k} \tag{19}$$

During valid structured JSON generation, emitting a structural delimiter token $w_{\text{struct}} \in \{\{, \}, ", :, ,\}$ requires that its logit exceeds the logits of competing conversational tokens $w_{\text{conv}} \in \{\text{"Here"}, \text{"Sure"}, \text{"JSON"}, \text{"The"}\}$ by a positive logit margin δz_t:

$$\delta z_t = z_{t,\text{struct}} - z_{t,\text{conv}} > 0 \tag{20}$$

Under dynamic INT8 quantization, matrix operations inherit weight quantization error $\mathbf{E}_W$ and dynamic activation error $\mathbf{E}_h$, producing perturbed logits $\hat{\mathbf{z}}_t$:

$$\hat{\mathbf{z}}_t = (\mathbf{W}_{\text{head}} + \mathbf{E}_W)(\mathbf{h}_t + \mathbf{E}_h) = \mathbf{z}_t + \epsilon_t \tag{21}$$

where $\epsilon_t = \mathbf{W}_{\text{head}} \mathbf{E}_h + \mathbf{E}_W \mathbf{h}_t + \mathbf{E}_W \mathbf{E}_h$ is the net logit noise vector. The perturbed logit margin $\widehat{\delta z}_t$ becomes:

$$\widehat{\delta z}_t = \hat{z}_{t,\text{struct}} - \hat{z}_{t,\text{conv}} = (z_{t,\text{struct}} - z_{t,\text{conv}}) + (\epsilon_{t,\text{struct}} - \epsilon_{t,\text{conv}}) \tag{22}$$

A **Logit Flip Failure** occurs whenever the noise difference exceeds the unquantized logit margin:

$$(\epsilon_{t,\text{conv}} - \epsilon_{t,\text{struct}}) > \delta z_t \implies \widehat{\delta z}_t < 0 \tag{23}$$

When $\widehat{\delta z}_t < 0$, the quantized model emits conversational token w_{conv} instead of w_{struct}, triggering a structural syntax violation.

4.2 Autoregressive Error Cascades

Logit flips during structural generation exhibit compounding autoregressive feedback. Once an unexpected token (e.g., ‘"Sure"‘) is selected at step t, it is appended to the KV cache context at step $t + 1$. Because the model's pre-training distribution strongly associates conversational text with natural language prose rather than valid JSON syntax, the conditional probability distribution $P(w_{t+1} \mid x_{<t}, \text{"Sure"})$ shifts dramatically away from structural delimiters. Consequently, a single quantization-induced logit flip cascades into severe structural departure, leading to unparseable conversational text or raw schema echoing.

5 Experimental Methodology & Benchmark Architecture

5.1 Model Selection and Hardware Configuration

We evaluate two representative small language models from the Qwen2.5 series [?]:

- Qwen2.5-0.5B-Instruct: 0.49B parameters, 24 transformer layers, 14 query attention heads, 2 key/value heads (Grouped Query Attention), hidden dimension $d = 896$, intermediate FFN dimension 4864.
- Qwen2.5-1.5B-Instruct: 1.54B parameters, 28 transformer layers, 12 query attention heads, 2 key/value heads (Grouped Query Attention), hidden dimension $d = 1536$, intermediate FFN dimension 8960.

All experiments were executed on a workstation powered by an Intel Xeon CPU system equipped with 15.44 GB RAM, running Ubuntu 22.04 LTS, PyTorch 2.2.1, and Hugging Face Transformers 4.38.2.

5.2 Quantization Implementation

For each model scale, two precision variants were constructed:

1) FP16 Baseline: Loaded in full 16-bit IEEE floating-point precision (‘torch.float16‘).
2) Dynamic INT8 Variant: Generated using PyTorch's ‘$\text{quantize}_d ynamic‘ inter face target inglinear layers(‘torch.nn.$
$channel to signed 8 - bit integers(‘qint8‘), while activations are d$
$tensor to unsigned 8 - bit integers(‘quint8‘) during CPU forward passes.$

5.3 Benchmark Dataset & Complexity Tiering

We constructed a evaluation benchmark comprising $N = 100$ deterministic JSON extraction tasks. Prompts are stratified across three structural complexity tiers:

- Tier 1: Primitive Flat ($N_1 = 35$): Single-level objects containing primitive scalar data types (string, integer, boolean).
- Tier 2: Nested Object ($N_2 = 35$): Multi-level hierarchical objects containing nested sub-structures and deeply scoped key paths.
- Tier 3: Typed Array ($N_3 = 30$): Complex schema specifications containing arrays of structured objects with strict internal type constraints.

5.4 Decoding Parameters

To eliminate sampling variance and isolate precision-induced formatting drift, all inference executions enforced locked greedy decoding settings:

- Temperature: $T = 0.0$
- Top-p (Nucleus Sampling): $p = 1.0$
- Random Seed: Locked at 42
- Maximum Generated Tokens: Enforced at 512 tokens.

[h]

Require: Model $\mathcal{M}$, Benchmark Dataset $\mathcal{D} = \{(x_i, \mathcal{S}_i)\}_{i=1}^{N}$
Ensure: Pass Rates $P_{\text{raw}}, P_{\text{scrubbed}}$, Drift $D_{\text{formatting}}$

$c_{\text{raw}} \leftarrow 0$, $c_{\text{scrubbed}} \leftarrow 0$
for each $(x_i, \mathcal{S}_i) \in \mathcal{D}$ do
  $y_i \leftarrow \mathcal{M}.\text{generate}(x_i, T = 0.0, \text{seed} = 42)$
  if JSON.validate$(y_i, \mathcal{S}_i)$ == TRUE then
    $c_{\text{raw}} \leftarrow c_{\text{raw}} + 1$
    $c_{\text{scrubbed}} \leftarrow c_{\text{scrubbed}} + 1$
  else
    $y_i' \leftarrow \text{RegexScrubMarkdown}(y_i)$
    if JSON.validate$(y_i', \mathcal{S}_i)$ == TRUE then
      $c_{\text{scrubbed}} \leftarrow c_{\text{scrubbed}} + 1$
    end if
  end if
end for
$P_{\text{raw}} \leftarrow c_{\text{raw}}/N$
$P_{\text{scrubbed}} \leftarrow c_{\text{scrubbed}}/N$
Compute $D_{\text{formatting}}$ using Eq. (??)
return $P_{\text{raw}}, P_{\text{scrubbed}}, D_{\text{formatting}}$

6 Empirical Results & Comparative Analysis

6.1 Main 2 × 2 Comparative Matrix

Table ?? summarizes the primary empirical results across all four experimental conditions.

The empirical measurements demonstrate severe precision-induced formatting breakdown:

1) FP16 Baselines Achieve Perfect Structural Compliance: Both Qwen2.5-0.5B and Qwen2.5-1.5B achieved 100.0% strict raw (P_{raw}) and scrubbed (P_{scrubbed}) pass rates in FP16 precision, proving that unquantized small language models possess robust inherent JSON extraction capabilities.
2) Catastrophic Structural Formatting Drift in INT8: Dynamic INT8 CPU quantization reduces strict raw pass rates (P_{raw}) to 0.0% for both models. Regex markdown scrubbing recovers a meager 7.0% pass rate for Qwen2.5-0.5B ($D_{\text{formatting}} = -93.0\%$) and fails entirely for Qwen2.5-1.5B ($P_{\text{scrubbed}} = 0.0\%$, $D_{\text{formatting}} = -100.0\%$).
3) Statistical Significance: Pooled two-proportion Z-tests yield $Z = -13.1845$ ($p < 10^{-15}$) for 0.5B and $Z = -14.1421$ ($p < 10^{-15}$) for 1.5B, confirming that structural degradation is highly significant and not attributable to sampling noise.

6.2 Refutation of Parameter Capacity Buffering

A widespread assumption in neural network compression literature is that increasing parameter count provides structural redundancy that buffers the architecture against quantization degradation. Our empirical results directly refute this hypothesis for structural schema compliance.

As detailed in Table ??, increasing parameter capacity by 3.1× (from 0.5B to 1.5B) failed to buffer the model against formatting drift. In fact, Qwen2.5-1.5B INT8 experienced total structural collapse (0.0% scrubbed pass rate), performing worse than Qwen2.5-0.5B INT8 (7.0% scrubbed pass rate). This confirms that structural token routing relies on precise numerical coordinate alignment in attention space, which is fundamentally destroyed by uncalibrated uniform dynamic quantization regardless of parameter scale in sub-2B architectures.

6.3 Stratified Complexity Analysis

Table ?? presents compliance pass rates broken down across schema complexity tiers.

The stratified evaluation reveals a strict complexity threshold:

- In Qwen2.5-0.5B INT8, successful generations (7.0% overall scrubbed pass) occurred exclusively within **Tier 1 (Primitive Flat)** schemas, where the model achieved a 20.0% scrubbed pass rate.
- For **Tier 2 (Nested Object)** and **Tier 3 (Typed Array)** schemas, compliance dropped to 0.0% across both quantized models. Multi-level nesting and array scoping require sustained multi-step state tracking, compounding quantization noise at every structural boundary.

7 Diagnostic Failure Taxonomy & Qualitative Case Studies

To establish a diagnostic taxonomy of structural failure modes, we systematically categorized all 193 failure instances recorded across the INT8 evaluation logs. As shown in Table ??, failures fall into three distinct categories.

7.1 Qualitative Analysis of Error Classes

7.1.1 SYNTAX_ERROR (67.9% of Failures)

The dominant failure mode across both quantized models was the generation of unparseable strings (67.9%). Qualitative inspection revealed three primary manifestations:

1) Conversational Preambles: The quantized model emits conversational introductory text (e.g., "Here

TABLE 1: Main 2 × 2 Empirical Matrix: Structural Compliance Pass Rates, Relative Formatting Drift ($D_{\text{formatting}}$), Test Statistics (Z), p-values, and Statistical Significance ($N = 100$ Prompts per Condition).

Model Architecture	Precision	P_{raw} (%)	P_{scrubbed} (%)	Semantic Acc (%)	$D_{\text{formatting}}$ (%)	Z-Statistic	p-value	Stat. Significa
Qwen2.5-0.5B-Instruct	FP16	100.0	100.0	94.0	Baseline	–	–	–
Qwen2.5-0.5B-Instruct	Dynamic INT8	0.0	7.0	0.0	-93.0	-13.1845	$< 10^{-15}$	True ($p < 0.0$
Qwen2.5-1.5B-Instruct	FP16	100.0	100.0	100.0	Baseline	–	–	–
Qwen2.5-1.5B-Instruct	Dynamic INT8	0.0	0.0	0.0	-100.0	-14.1421	$< 10^{-15}$	True ($p < 0.0$

TABLE 2: Stratified Structural Compliance Pass Rates across Schema Complexity Tiers.

Schema Complexity Tier	Sample N	FP16 Pass (%)	INT8 Pass (%)	Delta Δ (%)
Qwen2.5-0.5B-Instruct				
Tier 1: Primitive Flat	35	100.0	20.0	-80.0
Tier 2: Nested Object	35	100.0	0.0	-100.0
Tier 3: Typed Array	30	100.0	0.0	-100.0
Qwen2.5-1.5B-Instruct				
Tier 1: Primitive Flat	35	100.0	0.0	-100.0
Tier 2: Nested Object	35	100.0	0.0	-100.0
Tier 3: Typed Array	30	100.0	0.0	-100.0

TABLE 3: Diagnostic Failure Taxonomy Breakdown ($N = 193$ Total Failure Instances).

Failure Category	Primary Operational Manifestation	Count	Share (%)
SYNTAX_ERROR	Unparseable JSON, conversational chatter, code fences, truncation	131	67.9%
SCHEMA_FIELD_MISMATCH	Raw schema specification echoing, missing mandatory keys	61	31.6%
SCHEMA_TYPE_MISMATCH	Valid JSON syntax with incorrect primitive value data types	1	0.5%
Total		193	100.0%

is the extracted JSON payload:") prior to opening the JSON brace, breaking strict raw parsers.

2) Unclosed Code Fences: The model opens a markdown code block (“‘json) but terminates generation prematurely without emitting closing fences or closing brackets.
3) Conversational Trailing Disclaimers: The model generates a valid JSON block followed immediately by conversational summary bullets or explanatory disclaimers.

7.1.2 SCHEMA_FIELD_MISMATCH (31.6% of Failures)

The second most prevalent failure mode was structural field mismatch (31.6%). The primary manifestation was **Schema Echoing**: instead of populating data instances matching requested schema keys, quantized models frequently echoed the raw JSON Schema meta-definition (e.g., repeating "type": "object", "properties": {...}) verbatim.

7.1.3 SCHEMA_TYPE_MISMATCH (0.5% of Failures)

Type mismatch errors were exceedingly rare (0.5%). When quantized models successfully produced valid JSON syntax with correct key names, the primitive data types of the values almost always matched ground truth. This confirms that quantization degrades structural syntax routing rather than underlying semantic concept representation.

8 Threats to Validity & Protocol Scope

To ensure scientific rigor, we explicitly document the protocol scope and limitations of our findings:

1) Quantization Scheme Boundaries: Our empirical results apply specifically to **PyTorch uncalibrated dynamic CPU per-tensor INT8 quantization** (‘torch.ao.quantization.quantize$_dynamic$‘).$Theydonotextendau$ $guidedstaticpost-trainingquantization(e.g., GPTQ$ $1001[?], AWQ1001[?]), staticblock-$ $wisequantization(GGUFk-$ $quants), orquantizationformatswithexplicitoutlierpreserva$ $1001[?])$.
1) Decoding Paradigm Boundaries: Experiments were restricted to **unconstrained autoregressive decoding**. Implementing schema-constrained decoding (e.g., Outlines [?]) enforces grammar token masking at inference time, guaranteeing 100% syntactic compliance regardless of model precision.
2) Sample Size & Model Generalizability: While $N = 100$ prompts provided conclusive statistical power ($p < 10^{-15}$), future studies should evaluate additional model families (e.g., Llama-3, Mistral) and domain-specific schemas (e.g., medical records, financial compliance).

9 Prioritized Mitigation Strategies

Based on our empirical analysis, we rank three deployment interventions by computational cost and structural information gain:

1) Intervention 1: Schema-Constrained Decoding (Cost: Low | Gain: Immediate High): Enforce token masking via context-free grammar state machines (e.g., Outlines [?]) during CPU inference. This completely eliminates SYNTAX_ERROR and SCHEMA_FIELD_MISMATCH without requiring weight retraining.
2) Intervention 2: Outlier-Aware Block-Wise Quantization (Cost: Medium | Gain: High): Replace dynamic per-tensor CPU quantization with static GGUF Q8_0 or AWQ quantization featuring per-channel activation scaling, mitigating attention outlier clipping.
3) Intervention 3: Structure-Aware Quantization Calibration (Cost: High | Gain: High): Incorporate structured JSON extraction prompts into calibration datasets during static post-training quantization, preserving attention weight sensitivity over structural tokens.

10 Conclusion

In this paper, we introduced $D_{\text{formatting}}$ to quantify quantization-induced structural formatting collapse in small language models. Our empirical investigation of Qwen2.5-0.5B and Qwen2.5-1.5B reveals that dynamic INT8 CPU quantization triggers catastrophic formatting drift (-93.0% to -100.0%, $p < 10^{-15}$), refuting the parameter capacity buffering hypothesis. Diagnostic evaluation indicates that unparseable syntax errors (67.9%) and schema definition echoing (31.6%) dominate operational failures. We conclude that unconstrained dynamic INT8 quantization is unusable for structured data extraction in production SLM microservices, necessitating grammar-constrained decoding or outlier-aware block-wise quantization.

Appendix

.1 Proof of Test Statistic Bound under Complete Collapse

Proof. Consider the two-proportion pooled Z-statistic defined in Eq. (??):

$$Z = \frac{P_2 - P_1}{\sqrt{\bar{p}(1-\bar{p})\left(\frac{1}{N_1} + \frac{1}{N_2}\right)}} \tag{24}$$

When $N_1 = N_2 = N$, $P_1 = 1.0$ (FP16 baseline pass rate), and $P_2 = 0.0$ (INT8 quantized pass rate), the pooled success proportion $\bar{p}$ evaluates to:

$$\bar{p} = \frac{1.0 + 0.0}{2} = 0.5 \tag{25}$$

The denominator (Standard Error) evaluates to:

$$\text{SE} = \sqrt{0.5 \times (1 - 0.5) \times \frac{2}{N}} = \sqrt{0.25 \times \frac{2}{N}} = \sqrt{\frac{1}{2N}} = \frac{1}{\sqrt{2N}} \tag{26}$$

Thus, the test statistic Z is exact:

$$Z = \frac{0.0 - 1.0}{1/\sqrt{2N}} = -\sqrt{2N} \tag{27}$$

For sample size $N = 100$:

$$Z = -\sqrt{200} \approx -14.1421356 \tag{28}$$

This establishes the exact mathematical lower bound for test statistic Z under complete 100% to 0% structural pass collapse across equal sample sizes. □

.2 Exemplar Schema Complexity Tier Specifications

Listing ?? demonstrates a representative Tier 1 Primitive Flat schema specification used in our benchmark dataset.

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "UserScalarProfile",
  "type": "object",
  "properties": {
    "user_id": { "type": "integer" },
    "username": { "type": "string" },
    "is_active": { "type": "boolean" },
    "account_balance": { "type": "number" }
  },
  "required": ["user_id", "username", "is_active", "
      account_balance"],
  "additionalProperties": false
}
```

Listing 1: Exemplar Tier 1 Primitive Flat Schema Specification.

References

[1] L. Zheng, W.-L. Chiang, S. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. Xing et al., "Judging llm-as-a-judge with mt-bench and chatbot arena," Advances in Neural Information Processing Systems, vol. 36, pp. 46 595–46 623, 2023.

[2] T. Schick, J. Dwivedi-Yu, R. Dessì, S. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cattedra, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," Advances in Neural Information Processing Systems, vol. 36, pp. 68 539–68 551, 2023.

[3] M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. Baalen, and T. Blankevoort, "A white paper on neural network quantization," arXiv preprint arXiv:2106.08295, 2021.

[4] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "Llm.int8(): 8-bit matrix multiplication for transformers at scale," Advances in Neural Information Processing Systems, vol. 35, pp. 30 318–30 332, 2022.

[5] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "Gptq: Accurate post-training quantization for generative pre-trained transformers," in International Conference on Learning Representations (ICLR), 2023.

[6] J. Lin, J. Tang, H. Tang, S. Yang, X. Dang, and S. Han, "Awq: Activation-aware weight quantization for llm compression and acceleration," in Proceedings of the MLSys Conference, 2024.

[7] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "Smoothquant: Accurate and efficient post-training quantization for large language models," in International Conference on Machine Learning (ICML), 2023, pp. 38 087–38 099.

[8] A. Yang, B. Yang, B. Hui, B. Zheng, B. Yu, C. Zhou, C. Li, C. Li, G. Liu, H. Huang et al., "Qwen2.5 technical report," arXiv preprint arXiv:2409.12186, 2024.

[9] B. T. Willard and R. Louf, "Efficient guided generation for large language models," arXiv preprint arXiv:2307.09702, 2023.

[10] W. Shao, M. Chen, Z. Zhang, P. Xu, L. Zhao, Z. Li, K. Zhang, and C. Change Loy, "Omniquant: Omnidirectional calibrated post-training quantization for large language models," in International Conference on Learning Representations (ICLR), 2024.